

🖼️ SRI LANKAN FOOD RECOGNITION MODEL TRAINER 🖼️

Extend the model with your own food classes!

Tutorial Overview

Welcome! This notebook will teach you how to **extend** the Sri Lankan Food Recognition model with your own food classes using just **3 lines of code!**

What You'll Learn

-  How to use the `srilankan-food-trainer` library
-  How to add your own vegetable/food classes
-  How to train the model with minimal code
-  How to download and use your extended model

What You Need

1. Images of your food class (30-50 images recommended)

- Format: JPG, PNG, or JPEG
- Quality: Clear, well-lit photos
- Variety: Different angles and presentations

2. A folder structure like this:

```
potato_tempered/  
├── image1.jpg  
├── image2.jpg  
├── image3.jpg  
└── ...
```

3. Zip the folder before uploading

Time Required

- Setup: **2 minutes**
- Training: **30-60 minutes** (depending on number of images)
- Total: **~1 hour**

Let's Get Started!

Run the cells below **in order** from top to bottom.

▼ Quick Demo: See What's Possible!

Before we start, let's see what this library can do:

.....

WELCOME! 🙌

This tutorial shows you how to extend a pre-trained AI model to recognize **NEW** types of Sri Lankan food!

💡 **Key Concept: TRANSFER LEARNING**
Instead of training from scratch (which takes days), we'll use a pre-trained model and teach it **ONE** new class. This takes only **30-60 minutes!**

 What happens:

- ➡ Original model: Recognizes 8 classes
- ➡ You add: 1 new class (e.g., potato_tempered)
- ➡ Result: Model recognizes 9 classes (8 old + 1 new)

✨ The best part?
The model STILL recognizes the original 8 classes!
It doesn't forget them – it just learns something new!

```
print("="*70)
print("🎉 WELCOME TO SRI LANKAN FOOD TRAINER!")
print("="*70)

print("\n📖 In this tutorial, you'll:")
print("  1 Install the library (30 seconds)")
print("  2 Upload your food images (1 minute)")
print("  3 Train the model (30-60 minutes)")
print("  4 Download your extended model (30 seconds)")

print("\n💡 Pro Tip:")
print("  For best results, use 30-50 clear, well-lit images")
print("  of your food from different angles.")

print("\n🚀 Let's get started!\n")
```

```
=====
🎉 WELCOME TO SRI LANKAN FOOD TRAINER!
=====
```

📖 In this tutorial, you'll:

- 1 Install the library (30 seconds)
- 2 Upload your food images (1 minute)
- 3 Train the model (30-60 minutes)
- 4 Download your extended model (30 seconds)

💡 Pro Tip:
For best results, use 30-50 clear, well-lit images
of your food from different angles.

🚀 Let's get started!

📦 Step 1: Install the Library

First, we'll install the `srilankan-food-trainer` library. This library contains all the code needed to extend the model.

🕒 **Time:** ~30 seconds

▶ **Run the cell below:**

```
# =====
# STEP 1: INSTALL THE LIBRARY
# =====

"""
What's happening here?
-----
We're installing the 'srilankan-food-trainer' library from PyPI.
This is a custom Python package that simplifies model extension.

The '-q' flag means 'quiet mode' – it won't show all the installation details.

This installs:
✅ PyTorch (deep learning framework)
✅ torchvision (computer vision tools)
✅ huggingface_hub (to download pre-trained model)
✅ Helper utilities for training
"""

print("📦 Installing srilankan-food-trainer library...")
print("  This may take 20-30 seconds...\n")

!pip install -q srilankan-food-trainer

print("✅ Library installed successfully!")
print("\n💡 What we installed:")
print("  - FoodModelExtender: Main class for extending the model")
print("  - Pre-trained model downloader")
print("  - Training utilities")
print("\n▶ Ready for Step 2!")
```

-  Installing srilankan-food-trainer library...
This may take 20-30 seconds...
-  Library installed successfully!
-  What we installed:
 - FoodModelExtender: Main class for extending the model
 - Pre-trained model downloader
 - Training utilities
-  Ready for Step 2!

🔧 Step 2: Initialize the Model Extender

Now we'll create a `FoodModelExtender` object. This object will:

- Download the pre-trained model (if not already cached)
- Set up the training environment
- Prepare to accept your new food class

 **Time:** ~10 seconds

 **Run the cell below:**

```
# =====
# STEP 2: CREATE THE MODEL EXTENDER
# =====

"""
What's happening here?
-----
We're importing the FoodModelExtender class and creating an instance.

Parameters:
- verbose=True: Shows detailed progress messages (helpful for learning!)

Behind the scenes, this:
1. Checks if you have GPU (for faster training)
2. Sets up the model architecture
3. Prepares to download the pre-trained model when needed

The pre-trained model contains:
 8 vegetable classes across cooking states
 Learned features from 2000+ training images
 90%+ accuracy on validation data
"""

from srilankan_food_trainer import FoodModelExtender

print("="*70)
print("🔧 INITIALIZING FOOD MODEL EXTENDER")
print("="*70)

print("\n📦 Creating extender object...")
print("    This sets up the training environment\n")

# Create the extender with verbose output
extender = FoodModelExtender(verbose=True)

print("\n Extender initialized successfully!")
print("\n📊 Current status:")
print("    - Pre-trained model: Ready to download")
print("    - Original classes: 8 (will be preserved)")
print("    - New classes: 0 (you'll add yours next!)")
print("\n Ready for Step 3!")
```

```
=====
🔧 INITIALIZING FOOD MODEL EXTENDER
=====
```

```
 Creating extender object...
This sets up the training environment

 Sri Lankan Food Model Extender initialized!
Device: cuda
Base model: ranasinghehashini/srilankan-food-recognition

 Extender initialized successfully!

📊 Current status:
```

- Pre-trained model: Ready to download
- Original classes: 8 (will be preserved)
- New classes: 0 (you'll add yours next!)

 Ready for Step 3!

▼ Step 3: Upload Your Food Images

Time to add YOUR food class! You'll upload a ZIP file containing your images.

Preparation Checklist

Before running this cell, make sure you have:

-  **30-50 images** of your food (more is better!)
-  **Clear, well-lit photos** (avoid blurry images)
-  **Different angles** (top view, side view, close-up)
-  **Consistent subject** (all images should be the same food type)
-  **Organized in a folder** (e.g., `potato_tempered/`)
-  **Zipped the folder** (e.g., `potato_tempered.zip`)

Folder Structure Example

```
potato_tempered.zip
├── potato_tempered/      ← Folder name = class name
│   ├── img_001.jpg
│   ├── img_002.jpg
│   ├── img_003.jpg
│   └── ... (30-50 images total)
```

 **Time:** ~1-2 minutes (depending on file size)

 **Run the cell below and upload your ZIP file:**

```
# =====
# STEP 3: UPLOAD YOUR IMAGES
# =====

"""
What's happening here?
-----
We're using Google Colab's file upload feature to get your images.

Important naming convention:
- ZIP filename = class name
- Example: 'potato_tempered.zip' → class name is 'potato_tempered'
- Use underscores (_) not spaces
- Use lowercase for consistency

What happens after upload:
1. File is uploaded to Colab's temporary storage
2. We extract the class name from the filename
3. Ready to add to the model!

💡 Pro Tips:
- If upload is slow, check your internet connection
- If upload fails, try a smaller ZIP file
- Maximum recommended: 100 images per class
"""

from google.colab import files
import os

print("="*70)
print("
```

```

print(" - Good: potato_tempered.zip")
print(" - Good: eggplant_curry.zip")
print(" - Bad: My Food.zip (has space)")
print(" - Bad: images.zip (not descriptive)")

print("\n👉 Click 'Choose Files' below and select your ZIP file...\n")

# Upload file
uploaded = files.upload()

# Process uploaded file
if uploaded:
    filename = list(uploaded.keys())[0]
    file_size_mb = os.path.getsize(filename) / (1024 * 1024)

    print("\n" + "="*70)
    print("✅ FILE UPLOADED SUCCESSFULLY!")
    print("="*70)

    print(f"\n📁 File details:")
    print(f" - Filename: {filename}")
    print(f" - Size: {file_size_mb:.2f} MB")

    # Extract class name from filename
    class_name = filename.replace('.zip', '').replace('.ZIP', '')
    print(f" - Detected class name: '{class_name}'")

    print("\n📁 File saved to: /content/{0}".format(filename))
    print("\n👉 Ready for Step 4!")

else:
    print("\n❌ ERROR: No file uploaded!")
    print("\n💡 Please try again:")
    print(" 1. Run this cell again")
    print(" 2. Click 'Choose Files'")
    print(" 3. Select your ZIP file")
    raise Exception("Please upload your data file to continue")

```

📁 UPLOAD YOUR FOOD IMAGES

- 📁 Before uploading, make sure:
- ✅ Images are clear and well-lit
 - ✅ All images are of the SAME food type
 - ✅ You have 30-50 images (minimum 20)
 - ✅ Images are inside a folder
 - ✅ Folder is zipped

- 👉 Naming your file:
- Good: potato_tempered.zip
 - Good: eggplant_curry.zip
 - Bad: My Food.zip (has space)
 - Bad: images.zip (not descriptive)

👉 Click 'Choose Files' below and select your ZIP file...

Choose files potato_tempered.zip
potato_tempered.zip(application/zip) - 17903456 bytes, last modified: 09/02/2026 - 100% done
 Saving potato_tempered.zip to potato_tempered.zip

✅ FILE UPLOADED SUCCESSFULLY!

📁 File details:

- Filename: potato_tempered.zip
- Size: 17.07 MB
- Detected class name: 'potato_tempered'

📁 File saved to: /content/potato_tempered.zip

👉 Ready for Step 4!

📁 Step 4: Add Your Class to the Model

Now we'll add your food class to the model. This process will:

1. **Extract** your ZIP file
2. **Validate** your images (check format, count, etc.)
3. **Split** data into training (70%), validation (15%), and test (15%) sets

4. Prepare for training **Time:** ~10 seconds **Run the cell below:**

```
# =====
# STEP 4: ADD YOUR CLASS TO THE MODEL
# =====

"""
What's happening here?
-----
The add_class() method does several important things:

1. EXTRACTION:
    - Unzips your file
    - Finds the folder containing images

2. VALIDATION:
    - Counts images (needs minimum 20, recommends 30-50)
    - Checks image formats (JPG, PNG, JPEG)
    - Verifies images can be opened

3. DATA SPLITTING:
    - 70% for training (model learns from these)
    - 15% for validation (checks progress during training)
    - 15% for testing (final evaluation)

4. PREPARATION:
    - Organizes files in proper structure
    - Creates labels
    - Sets up data loaders

Parameters explained:
    - class_name: Name of your food class (from ZIP filename)
    - images_path: Path to your ZIP file
    - auto_extract: Automatically unzip the file (True/False)
"""

print("="*70)
print("📁 ADDING YOUR CLASS TO THE MODEL")
print("="*70)

print(f"\n🔄 Processing class: '{class_name}'")
print(f"\n📝 This will:")
print("    1. Extract your ZIP file")
print("    2. Count and validate images")
print("    3. Split into train/validation/test sets")
print("    4. Prepare for training\n")

# Get the file path
zip_path = f'/content/{filename}'

# Add class to the model
success = extender.add_class(
    class_name=class_name,
    images_path=zip_path,
    auto_extract=True # Automatically unzip
)

print("\n" + "="*70)

if success:
    print("✅ CLASS ADDED SUCCESSFULLY!")
    print("="*70)

    print(f"\n📊 Summary:")
    print(f"    - Class name: {class_name}")
    print(f"    - Original model classes: 8")
    print(f"    - Your new class: 1")
    print(f"    - Total classes after training: 9")

    print(f"\n💡 Important:")
    print("    The model will learn to recognize your new class")
    print("    WITHOUT forgetting the original 8 classes!")

    print(f"\n🚀 Ready for Step 5 (Training)!")

else:
    print("❌ FAILED TO ADD CLASS")
    print("="*70)
```

```
print("\n💡 Possible issues:")
print("  - Not enough images (need minimum 20)")
print("  - Images in wrong format (use JPG/PNG)")
print("  - ZIP structure is incorrect")
print("\n📄 Check the error message above for details")
raise Exception("Class addition failed. Please fix the issues and try again.")
```

📦 ADDING YOUR CLASS TO THE MODEL

🔄 Processing class: 'potato_tempered'

🕒 This will:

1. Extract your ZIP file
2. Count and validate images
3. Split into train/validation/test sets
4. Prepare for training

📦 Adding new class: potato_tempered
 Extracting /content/potato_tempered.zip...
 Found 37 images
 Splitting into train/val/test...
 ✅ Split complete:
 Train: 25 images
 Val: 5 images
 Test: 7 images

✅ CLASS ADDED SUCCESSFULLY!

📊 Summary:
 - Class name: potato_tempered
 - Original model classes: 8
 - Your new class: 1
 - Total classes after training: 9

💡 Important:
 The model will learn to recognize your new class
 WITHOUT forgetting the original 8 classes!

👉 Ready for Step 5 (Training)!

🏆 Step 5: Train the Model

This is where the magic happens! The model will learn to recognize your new food class.

🧠 What is Training?

Training is the process where the AI model:

1. **Looks** at your images
2. **Tries** to recognize patterns
3. **Makes** mistakes (initially)
4. **Learns** from those mistakes
5. **Improves** its accuracy over time

Think of it like teaching a child to recognize different foods - it takes examples and practice!

📊 Training Process

Epoch 1/50: Model sees all training images once → Accuracy: 65%
 Epoch 2/50: Model sees all training images again → Accuracy: 72%
 Epoch 3/50: Model sees all training images again → Accuracy: 78%
 ...
 Epoch 50/50: Final round → Accuracy: 90%

🕒 Expected Time

- With GPU: **30-45 minutes** ⚡ (faster)
- With CPU: **60-90 minutes** 🐢 (slower)

Colab usually provides a free GPU, so it should be faster!

📈 What You'll See

During training, you'll see:

- **Progress bar:** Shows which epoch we're on
- **Training loss:** Lower is better (model's mistakes)
- **Validation accuracy:** Higher is better (model's correctness)
- **Time per epoch:** How long each round takes

☕ Pro Tip

This is a good time to:

- Read about transfer learning 📖
- Watch the progress occasionally 👁️

▶ **Run the cell below to start training:**

```
# =====
# STEP 5: TRAIN THE MODEL
# =====

"""
What's happening here?
-----
This is the core of the tutorial – training the model!

The training process:

1. INITIALIZATION (first time only):
   - Downloads pre-trained model from Hugging Face
   - Loads the 8 original classes
   - Sets up the neural network

2. FINE-TUNING (each epoch):
   - Shows model your training images
   - Model makes predictions
   - Calculates how wrong the predictions are (loss)
   - Adjusts model weights to improve
   - Tests on validation images

3. MONITORING:
   - Tracks best accuracy
   - Saves best model version
   - Prevents overfitting

Training Parameters:
   - epochs: Number of times to see all training data (default: 50)
   - learning_rate: How fast to learn (default: 0.00001, very small!)
   - batch_size: Images processed together (default: configured in library)

Why such a low learning rate?
   We're fine-tuning, not training from scratch!
   Small changes preserve the original 8 classes.

Output metrics explained:
   - Train Loss: Lower = better (0.1 is great, 1.0 is okay)
   - Val Accuracy: Higher = better (90%+ is excellent, 80%+ is good)
   - Time: Depends on GPU availability
"""

print("="*70)
print("🚀 TRAINING THE MODEL")
print("="*70)

print("\n🤖 What will happen:")
print("  1. Download pre-trained model (first time only)")
print("  2. Load original 8 classes")
print("  3. Train for 50 epochs (~30-60 minutes)")
print("  4. Save best model automatically")

print("\n💡 Understanding the output:")
print("  - Epoch: One complete pass through all training images")
print("  - Loss: How wrong the model is (lower = better)")
print("  - Val Acc: Validation accuracy (higher = better)")
print("  - Target: 85-90% validation accuracy")

print("\n🕒 Estimated time:")
print("  - With GPU: 30-45 minutes ⚡")
print("  - With CPU: 60-90 minutes 🐢")

print("\n☕ This is a great time for a coffee break!")
print("  The notebook will continue running in the background.\n")
```



```

print("="*70)
print("🚀 STARTING TRAINING...")
print("="*70 + "\n")

# Train the model
# This will take 30-60 minutes depending on GPU availability
results = extender.train(
    epochs=50,          # Number of training rounds
    save_checkpoints=True # Save best model automatically
)

# Training complete! Show results
print("\n" + "="*70)
print("🎉 TRAINING COMPLETE!")
print("="*70)

print("\n📊 Final Results:")
print(f"✅ Best Validation Accuracy: {results['best_val_acc']:.2f}%")
print(f"🕒 Total Training Time: {results.get('total_time', 0)/60:.1f} minutes")
print(f"💾 Model saved automatically")

print("\n📈 Performance Guide:")
if results['best_val_acc'] >= 90:
    print("🌟 EXCELLENT! (90%+) - Your model is performing great!")
elif results['best_val_acc'] >= 85:
    print("✅ VERY GOOD! (85-90%) - Model should work well in practice.")
elif results['best_val_acc'] >= 80:
    print("👍 GOOD! (80-85%) - Decent performance, consider more training.")
else:
    print("⚠️ NEEDS IMPROVEMENT (<80%) - Consider:")
    print("    - Adding more images")
    print("    - Training for more epochs")
    print("    - Checking image quality")

print("\n💡 What your model can now do:")
print("✅ Recognize all 8 original classes (carrot, beans, pumpkin, etc.)")
print(f"✅ Recognize your new class: {class_name}")
print("✅ Total: 9 classes in one model!")

print("\n📁 Ready for Step 6 (Save & Download)!")

```


TRAINING THE MODEL

What will happen:

1. Download pre-trained model (first time only)
2. Load original 8 classes
3. Train for 50 epochs (~30-60 minutes)
4. Save best model automatically

Understanding the output:

- Epoch: One complete pass through all training images
- Loss: How wrong the model is (lower = better)
- Val Acc: Validation accuracy (higher = better)
- Target: 85-90% validation accuracy

Estimated time:

- With GPU: 30-45 minutes ✂
- With CPU: 60-90 minutes 🐢

This is a great time for a coffee break!

The notebook will continue running in the background.

STARTING TRAINING...

⚠ Pre-trained model not loaded. Downloading...

📦 Downloading pre-trained model...

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings>).
You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn()

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limit.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set

best_model.pth: 100%

5.89M/5.89M [00:00<00:00, 6.57MB/s]

✅ Model downloaded successfully!

Original classes: 8

- carrot_raw
- carrot_white_curry
- greenbeans_raw
- greenbeans_tempered
- greenbeans_white_curry
- pumpkin_raw
- pumpkin_red_curry
- pumpkin_white_curry

STARTING TRAINING

New classes: ['potato_tempered']

Epochs: 50

Device: cuda

🔔 Epoch 1/50

Training: 100%|████████████████████| 100/100 [00:13<00:00, 7.22it/s, loss=0.0000, acc=100.0%]

Train Loss: 0.0000

Train Acc: 100.00%

Val Acc: 100.00%

✅ New best! (100.00%)

🔔 Epoch 2/50

Training: 100%|████████████████████| 100/100 [00:11<00:00, 8.49it/s, loss=0.0000, acc=100.0%]

Train Loss: 0.0000

Train Acc: 100.00%

Val Acc: 100.00%

🔔 Epoch 3/50

Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.15it/s, loss=0.0000, acc=100.0%]

Train Loss: 0.0000

Train Acc: 100.00%

Val Acc: 100.00%

🔔 Epoch 4/50

Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.12it/s, loss=0.0000, acc=100.0%]

Train Loss: 0.0000

Train Acc: 100.00%

Val Acc: 100.00%

🔔 Epoch 5/50

Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.20it/s, loss=0.0000, acc=100.0%]

Train Loss: 0.0000

Train Acc: 100.00%

Val Acc: 100.00%

```
Epoch 6/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.19it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 7/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.23it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 8/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 7.99it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 9/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 7.90it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 10/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.10it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 11/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.18it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 12/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.25it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 13/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.33it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 14/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.19it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 15/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.11it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 16/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 7.90it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 17/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 7.94it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 18/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.25it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 19/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.21it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 20/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 7.87it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%
```

```
Epoch 21/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.00it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 22/50
Training: 100%|████████████████████| 100/100 [00:11<00:00, 8.40it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 23/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.13it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 24/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.08it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 25/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.07it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 26/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.09it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 27/50
Training: 100%|████████████████████| 100/100 [00:11<00:00, 8.38it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 28/50
Training: 100%|████████████████████| 100/100 [00:11<00:00, 8.77it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 29/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.20it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 30/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.15it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 31/50
Training: 100%|████████████████████| 100/100 [00:11<00:00, 8.41it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 32/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.21it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 33/50
Training: 100%|████████████████████| 100/100 [00:11<00:00, 8.37it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 34/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.29it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 35/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.07it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 36/50
Training: 100%|████████████████████| 100/100 [00:12<00:00, 8.07it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%
```

```
Epoch 36/50
Training: 100%|████████████████████████████████████████| 100/100 [00:11<00:00, 8.54it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 37/50
Training: 100%|████████████████████████████████████████| 100/100 [00:12<00:00, 8.27it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 38/50
Training: 100%|████████████████████████████████████████| 100/100 [00:12<00:00, 8.11it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 39/50
Training: 100%|████████████████████████████████████████| 100/100 [00:11<00:00, 8.42it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 40/50
Training: 100%|████████████████████████████████████████| 100/100 [00:12<00:00, 8.11it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 41/50
Training: 100%|████████████████████████████████████████| 100/100 [00:11<00:00, 8.37it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 42/50
Training: 100%|████████████████████████████████████████| 100/100 [00:12<00:00, 7.87it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 43/50
Training: 100%|████████████████████████████████████████| 100/100 [00:11<00:00, 8.42it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 44/50
Training: 100%|████████████████████████████████████████| 100/100 [00:12<00:00, 8.03it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 45/50
Training: 100%|████████████████████████████████████████| 100/100 [00:12<00:00, 8.10it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 46/50
Training: 100%|████████████████████████████████████████| 100/100 [00:12<00:00, 8.23it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 47/50
Training: 100%|████████████████████████████████████████| 100/100 [00:12<00:00, 8.23it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 48/50
Training: 100%|████████████████████████████████████████| 100/100 [00:12<00:00, 8.24it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 49/50
Training: 100%|████████████████████████████████████████| 100/100 [00:12<00:00, 8.06it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%

Epoch 50/50
Training: 100%|████████████████████████████████████████| 100/100 [00:12<00:00, 7.84it/s, loss=0.0000, acc=100.0%]
Train Loss: 0.0000
Train Acc: 100.00%
Val Acc: 100.00%
```

🔥 TRAINING COMPLETE!

Step 6: Save and Download Your Model

Congratulations! Your model is trained. Now let's save it and download it to your computer.

🔥 TRAINING COMPLETE!

What Gets Saved

📊 Final Results:

The saved file contains:

- ✓ Best Validation Accuracy: 100.00%
- ✓ Total Training Time: 14.4 minutes
- ✓ Model weights (the learned parameters)
- ✓ Model saved automatically
- ✓ Original 8 classes (preserved knowledge)
- ✓ Your new class (newly learned)
- ✓ Performance Guide: EXCELLENT! (90%+) - Your model is performing great!
- ✓ Training history (accuracy, loss, etc.)
- ✓ Metadata (model configuration, etc.)
- ✓ Recognize all 8 original classes (carrot, beans, pumpkin, etc.)
- ✓ Recognize your new class: potato_tempered
- ✓ Total 9 classes in one model!

File Details

- **Filename:** extended_model.pth (download)!
- **Size:** ~10-15 MB
- **Format:** PyTorch checkpoint

What's Next

After downloading, you can:

1. Use it in a Python application
2. Convert to TFLite for mobile apps
3. Deploy to a web service
4. Share with others

🕒 **Time:** ~30 seconds

▶ **Run the cell below:**

```
# =====
# STEP 6: SAVE AND DOWNLOAD YOUR MODEL
# =====

"""
What's happening here?
-----

We're saving the trained model and downloading it to your computer.

The save() method:
1. Packages the model weights
2. Includes original class information
3. Includes new class information
4. Adds training metadata
5. Saves as a .pth file (PyTorch format)

File structure:
{
  'model_state_dict': [neural network weights],
  'original_classes': [8 class names],
  'new_classes': [your class name],
  'base_model': 'ranasinghehashini/srilankan-food-recognition',
  'training_history': [loss, accuracy per epoch],
  'timestamp': [when trained]
}

You can load this later with:
checkpoint = torch.load('extended_model.pth')
model.load_state_dict(checkpoint['model_state_dict'])
"""

print("="*70)
print("📁 SAVING YOUR EXTENDED MODEL")
print("="*70)

print("\n📁 Saving model checkpoint...")
print("   This includes:")
print("   ✓ Trained model weights")
print("   ✓ All 9 class labels")
print("   ✓ Training history")
print("   ✓ Metadata\n")
```